

05 — Python dunder methods and special names

The question

What does dunder mean, and how do I use names such as `__init__`, `__len__`, and `__name__` in Python?

Dunder is short for double underscore. Names such as `__init__` have two underscores at each end. Distinguish special methods from special attributes or module variables: `__len__` is a method, whereas a module's `__name__` is a string.

1. Connect familiar operations to special methods

When using an object, normally write `len(x)` or `x[0]`. When writing your own class, implement the appropriate special methods to support those operations. These method names are part of Python's protocols; do not invent new dunder names for ordinary helpers.

Operation	Special method	Purpose
Initialize an instance	<code>__init__</code>	Set initial attributes
<code>print(x)</code> , <code>str(x)</code>	<code>__str__</code>	Human-readable text
<code>repr(x)</code>	<code>__repr__</code>	Debugging representation
<code>len(x)</code>	<code>__len__</code>	Number of items
<code>x[0]</code>	<code>__getitem__</code>	Index or key lookup
<code>x + y</code>	<code>__add__</code>	Addition behavior
<code>x == y</code>	<code>__eq__</code>	Equality comparison
<code>iter(x)</code>	<code>__iter__</code>	Supply an iterator
<code>x()</code>	<code>__call__</code>	Make an instance callable

This table is a lookup aid, not a literal expansion of every expression. Operators may involve reflected methods or fallbacks. For example, addition can also consider `__radd__`. Implicit special-method lookup generally uses the object's type, so attaching a method only to an individual instance is not a reliable way to support `len(instance)`.

2. A small class for scenario profits

The numbers below are illustrative rounded profits, not a new Case 1 calculation. Save or run [the complete example](#); it requires Python 3.9+ and no external packages.

```
class ProfitSamples:
    def __init__(self, values):
        self.values = list(values)

    def __len__(self):
        return len(self.values)

    def __getitem__(self, index):
        return self.values[index]

    def __repr__(self):
        return f"ProfitSamples({self.values!r})"
```

Now create an instance and inspect it:

```
profits = ProfitSamples([0.61, 0.41, 0.71])
print(len(profits))      # 3
print(profits[0])        # 0.61
print(profits)           # ProfitSamples([0.61, 0.41, 0.71])
```

Read our class one method at a time:

- `__init__` stores the supplied values in a list. It initializes an already created instance; object creation is handled by `__new__`. An initializer must not return a non-None value.
- `self` refers to the current instance. It is a conventional parameter name, not a keyword or a dunder name. Python supplies it when a bound method is called.
- `__len__` delegates to the stored list, so this instance reports three items.
- `__getitem__` delegates to that list, so index 0 returns 0.61. In this implementation, `profits[:2]` returns a list, not another `ProfitSamples` instance.
- `__repr__` includes the class name and stored values. The `!r` in the f-string uses `repr(self.values)`. Our class does not override `__str__`, so printing it falls back to this representation.

In Spyder, put a breakpoint on `return len(self.values)` and debug the script. When `len(profits)` reaches the breakpoint, inspect `type(self)`, `self.values`, and `type(self.values)`. This connects the custom instance with the ordinary list it contains.

3. The script entry point: `__name__`

Our example ends with:

```
def main():
    profits = ProfitSamples([0.61, 0.41, 0.71])
    print(len(profits))
    print(profits[0])
    print(profits)

if __name__ == "__main__":
    main()
```

Here `__name__` is a module variable, not a method. Running the file directly gives it the value `"__main__"`, so the demonstration runs. Loading it as an imported module with a different name leaves the guarded call unexecuted. Importing still executes other top-level statements, including class and function definitions; the guard protects only its own block.

From the repository root:

```
python examples/05_dunder_methods.py
```

Expected output:

```
3
0.61
ProfitSamples([0.61, 0.41, 0.71])
```

For a normal file named `experiment.py`, `import experiment` sets that module's `__name__` to `"experiment"`; package imports can use a dotted name such as `"package.experiment"`. The handbook example's numeric filename is convenient for ordering, so use `importlib` if you want to import it by path rather than writing a normal import statement starting with a digit.

4. Similar underscore patterns have different meanings

Pattern	Meaning
<code>value</code>	Ordinary name
<code>_value</code>	Convention for internal use, not access control
<code>__value</code> inside a class	Name mangling to reduce subclass name collisions
<code>__value__</code>	Form used for documented special names

For example, a class `Demo` with `self.__value = 1` normally stores it under `__Demo__value`. This is not a security boundary. `__init__` has trailing double underscores and is not a private method.

5. What to learn first

Start with `__init__`, `__repr__`, `__len__`, `__getitem__`, and the `__name__` guard. You do not need a custom class just to calculate profits with `NumPy`. A class becomes useful when related data and operations should travel together.

Try predicting `profits[-1]`, `profits[:2]`, and `type(profits.values)`. For this class, the answers are `0.71`, `[0.61, 0.41]`, and `list`. The built-in list provides the indexing rules because our implementation delegates to it.

Sources

See Python's [special-method reference](#)^[1] for protocol details and its [top-level code environment guide](#)^[2] for the entry-point guard. The `ProfitSamples` example and inspection exercise are specific to this handbook.

References

1. <https://docs.python.org/3/reference/datamodel.html#special-method-names>
2. https://docs.python.org/3/library/__main__.html